🚀 Онбординг: как запустить проект

Этот проект — бэкенд на **Django**. Если ты раньше работал с Kotlin, Rust или другими языками, не пугайся: Django устроен логично, но есть свои концепции (миграции, приложения, `manage.py`). Ниже — пошаговая инструкция и разбор того, что где лежит.

1. Что должно быть установлено

- **Python 3.11 или новее** (проверь: `python --version`). Если у тебя 3.10 или ниже — поставь свежую версию через python.org или `pyenv`.
- **Git** — он у тебя уже есть.
- Всё остальное поставим ниже.

2. Клонирование и переход в бэкенд

Клонируй репозиторий (как обычно) и перейди в папку `backend/`:

```
git clone <url-репозитория>
cd <репозиторий>/backend
```

Именно в `backend/` лежит `manage.py` — с ним мы будем работать. Все команды из этого гайда выполняются **из папки `backend/`**.

3. Виртуальное окружение (venv)

Виртуальное окружение изолирует зависимости проекта от системного Python. Это аналог того, как в Kotlin/Gradle зависимости привязаны к проекту. Без venv все пакеты поставятся «глобально» и рано или поздно перепутаются.

```
# Создаём окружение (один раз)
python -m venv venv

# Активируем (нужно КАЖДЫЙ раз при работе с проектом)
# Linux/macOS:
source venv/bin/activate
# Windows (PowerShell):
.\venv\Scripts\Activate.ps1
# Windows (cmd):
venv\Scripts\activate.bat
```

После активации в начале строки терминала появится `(venv)`. Проверь: `python --version` теперь указывает на Python внутри venv.

💡 **Не коммить `venv/`** — он в `.gitignore`, и правильно. У каждого разработчика своё окружение.

4. Установка зависимостей

```
pip install --upgrade pip
pip install -r requirements.txt
```

`requirements.txt` — список пакетов с зафиксированными версиями. Это аналог `build.gradle` / `Cargo.toml` в мире Python. Все разработчики ставят **одни и те же версии**, чтобы «у меня работает, а у тебя нет» не случалось.

5. Миграции — что это и зачем

Миграция — это файл с инструкциями, как изменить структуру базы данных (создать таблицу, добавить колонку, удалить поле и т.д.).

В Django ты **не пишешь SQL руками**. Ты описываешь модели в Python (классы), а Django сам:

1. Сравнивает модели с текущим состоянием БД;
2. Генерирует файл миграции (что именно изменилось);
3. Применяет его к базе.

Это похоже на **версионирование схемы БД через git**: каждая миграция — коммит в истории схемы. У миграций есть номера и порядок, и они применяются последовательно.

Зачем это тебе сейчас: когда ты только что склонировал репу, у тебя нет локальной базы. Таблицы нужно создать — это делается применением миграций.

```
python manage.py migrate
```

Первый запуск создаст файл `db.sqlite3` (это база для разработки — её тоже не коммитят, она в `.gitignore`). Ты увидишь вывод вроде `Applying auth.0001_initial... OK` — это Django по шагам строит схему.

Команды, которые надо знать про миграции:

Команда	Что делает
`python manage.py makemigrations`	Сгенерировать новую миграцию **после изменения моделей** в коде
`python manage.py migrate`	Применить все неприменённые миграции к БД
`python manage.py showmigrations`	Показать список миграций и их статус (применена/нет)

⚠️ **Золотое правило:** изменил модель → `makemigrations` → `migrate`. Если склонировал репу и у тебя появились новые миграции от коллег → просто `migrate`.

6. Запуск dev-сервера

```
python manage.py runserver
```

Открой http://127.0.0.1:8000. Если видишь ракету Django — всё работает. 🚀

Сервер **автоматически перезагружается** при изменении `.py`-файлов — не надо останавливать и запускать заново (аналог `cargo watch` или hot reload во фронтенде). Для остановки — `Ctrl+C`.

7. Структура проекта: что где лежит

```
backend/
├── manage.py              ← CLI для всех команд (аналог cargo/gradle wrapper)
├── requirements.txt       ← зависимости проекта
├── db.sqlite3             ← локальная БД (в git НЕ попадает, создаётся миграциями)
├── config/                ← «пульт управления» всего проекта
│   ├── settings.py        ← конфигурация: БД, приложения, статика, middleware
│   ├── urls.py            ← корневая маршрутизация (главный "роутер")
│   ├── asgi.py            ← точка входа для WebSocket (нужна позже)
│   ├── wsgi.py            ← точка входа для HTTP в продакшене
│   └── __init__.py
└── staticfiles/           ← собранная статика для продакшена (в git НЕ попадает)
```

В Django есть два уровня организации: **проект** (`config/`) — общая конфигурация, и **приложения** (apps) — модули с логикой. Пока что приложений нет, они появятся позже (каждое будет отдельной папкой со своим `models.py`, `views.py` и т.д.).

8. Шпаргалка по командам

Все команды — из папки `backend/` с активированным `venv`:

```
python manage.py runserver          # запустить dev-сервер
python manage.py migrate            # применить миграции
python manage.py makemigrations     # создать миграцию после правок моделей
python manage.py createsuperuser    # создать админа для /admin
python manage.py shell              # интерактивная Python-консоль с Django
python manage.py startapp <name>    # создать новое приложение
```

9. Частые проблемы

- **`command not found: django-admin`** — не активировал venv, либо не установил зависимости.
- **`ModuleNotFoundError: No module named 'django'`** — то же самое: `pip install -r requirements.txt` внутри активированного venv.
- **`no such table: ...`** — забыл `python manage.py migrate`.
- **Сервер не перезагружается** — проверь, что редактируешь файлы внутри `backend/`, а не в кэше/старой папке.

- **`db.sqlite3` в `git status`** — не должно быть. Проверь, что `.gitignore` подтянулся (`git pull`).

10. Чек-лист «я готов к работе»

- [ ] Python 3.11+ установлен
- [ ] venv создан и активирован
- [ ] `pip install -r requirements.txt` прошёл без ошибок
- [ ] `python manage.py migrate` завершился успешно
- [ ] `python manage.py runserver` открывает ракету на `http://127.0.0.1:8000`
- [ ] Я понимаю, что такое миграция и зачем `makemigrations` / `migrate`
- [ ] Я знаю, что команды выполняются из `backend/`, а `venv` активируется каждую сессию

Если все пункты отмечены — можно приступать к первой задаче. 🎯